



Lab#9 – Procedures

Procedures (functions)

In x86 assembly, a procedure (also known as a subroutine or function) is a reusable block of code that performs a specific task. Procedures help make code more organized, modular, and easier to maintain. By breaking down a program into smaller procedures, you can reuse code and avoid redundancy. Each procedure can be called from different parts of the program and, when executed, performs its defined task before returning control to the calling code. In assembly, a procedure typically starts with a label (procedure name) and ends with a RET (return) instruction, which transfers control back to the instruction immediately following the call to the procedure. The CALL instruction is used to invoke a procedure, and it automatically pushes the address of the next instruction onto the stack, allowing RET to know where to return.

Important: If you use the PUSH instruction within a procedure, make sure to POP the corresponding value before reaching the RET instruction. Failing to do so can lead to an incorrect return address on the stack, causing the program to return to the wrong location or crash.

Example:

```
INCLUDE Irvine32.inc
```

```
.data
```

```
    resultMessage BYTE "The sum is: ", 0
```

```
.code
```

```
main PROC
```

```
    MOV EAX, 5           ; Load first number into EAX
```

```
    MOV EBX, 10          ; Load second number into EBX
```

```
    CALL Sum             ; Call the Sum procedure
```

```
    MOV EDX, OFFSET resultMessage
```

```
    CALL WriteString     ; Display the result message
```

```
    CALL WriteInt        ; Display the result in EAX
```

```
    CALL CrLf
```

```
    EXIT
```

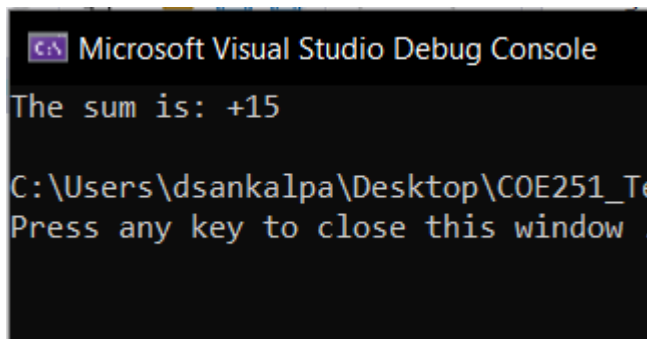
```
main ENDP
```

```

;-----
; Sum Procedure: Adds the values in EAX and EBX
; and returns the result in EAX.
;-----
Sum PROC
    ADD EAX, EBX      ; Add EAX and EBX
    RET              ; Return to the calling function
Sum ENDP

END main

```



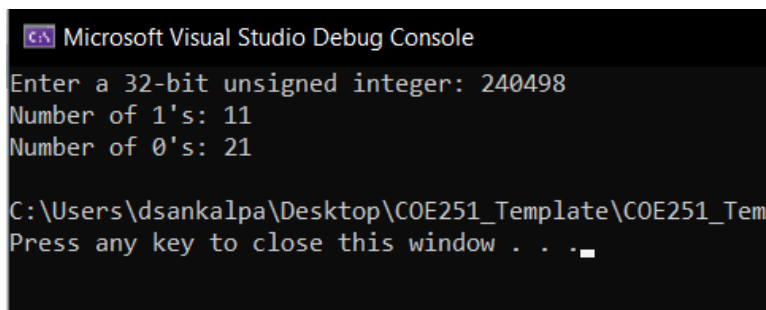
```

Microsoft Visual Studio Debug Console
The sum is: +15
C:\Users\dsankalpa\Desktop\COE251_Template\COE251_Template.asm
Press any key to close this window

```

Lab Work (Questions 1-4):

1. Write an assembly program that reads a 32-bit unsigned integer from the user and counts the number of 1's and 0's in its binary representation. Use the ROR (rotate right) instruction to rotate through each bit and check whether it is set.



```

Microsoft Visual Studio Debug Console
Enter a 32-bit unsigned integer: 240498
Number of 1's: 11
Number of 0's: 21
C:\Users\dsankalpa\Desktop\COE251_Template\COE251_Template.asm
Press any key to close this window

```

Solution:

TITLE Name/Purpose Here

INCLUDE Irvine32.inc

.data

```

; Your data goes here
prompt BYTE "Enter a 32-bit unsigned integer: ",0
ones BYTE "Number of 1's: ",0

```

```

        zeroes BYTE "Number of 0's: ",0
        zero  DWORD 0
        one   DWORD 0
.code
main PROC
    mov edx, OFFSET prompt
    call writestring
    call readdec

    mov ecx, 32
counting:
    ror eax, 1
    jnc iszero
    add one, 1
    jmp done
iszero:
    add zero, 1
done:
    loop counting

    mov edx, OFFSET ones
    call writestring
    mov eax, one
    call writedec

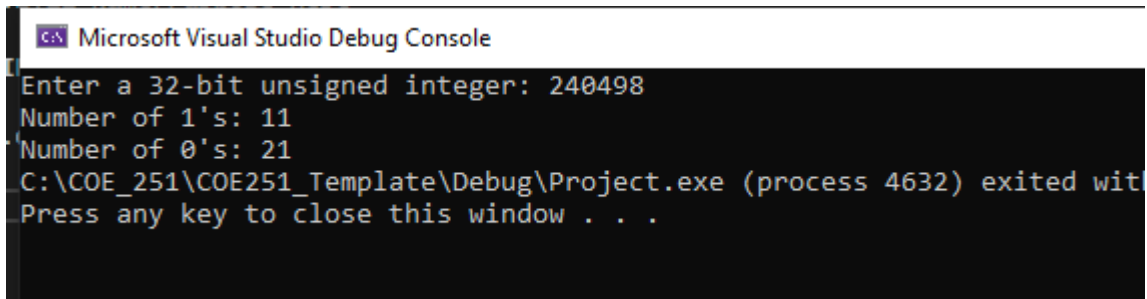
    call crlf

    mov edx, OFFSET zeroes
    call writestring
    mov eax, zero
    call writedec

    exit
main ENDP
END main

```

Screenshot:

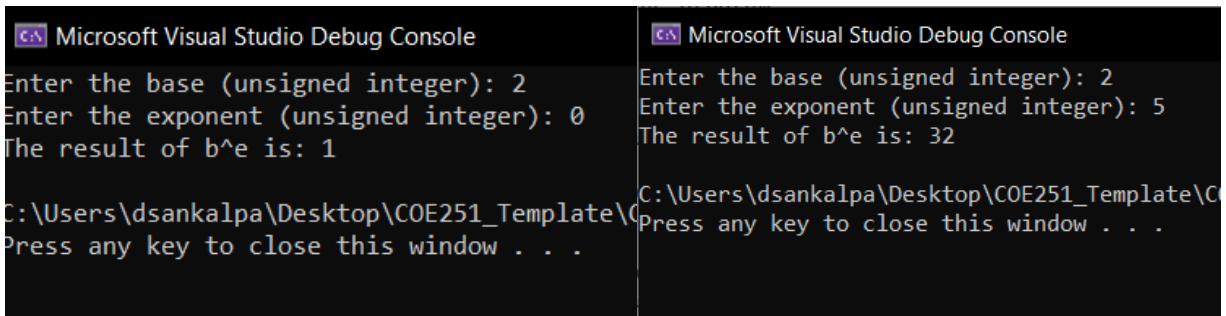


```

Microsoft Visual Studio Debug Console
Enter a 32-bit unsigned integer: 240498
Number of 1's: 11
Number of 0's: 21
C:\COE_251\COE251_Template\Debug\Project.exe (process 4632) exited with
Press any key to close this window . . .

```

- Write an assembly program that uses a procedure called **Power**. The procedure should take two unsigned integers as input: a base b and an exponent e , and it should calculate b^e (base raised to the power of exponent). **Display the result in the main program.**



Solution:

TITLE Name/Purpose Here

INCLUDE Irvine32.inc

.data

```
promptb BYTE "Enter the base (unsigned integer): ",0
prompte BYTE "Enter the exponent (unsigned integer): ",0
output BYTE "The result of b^e is: ",0
b DWORD 0
e DWORD 0
result DWORD 1
```

.code

main PROC

```
mov edx, OFFSET promptb
call writestring
call readdec
mov b, eax
```

```
mov edx, OFFSET prompte
call writestring
call readdec
mov e, eax
```

```
call Power
```

```
mov edx, OFFSET output
call writestring
```

```
mov eax, result
call writedec
```

```
exit
```

main ENDP

Power PROC

```
push eax
push ecx
```

```
mov eax, 1
mov ebx, e
cmp ebx, 0
je iszero
```

```
jmp notzero
```

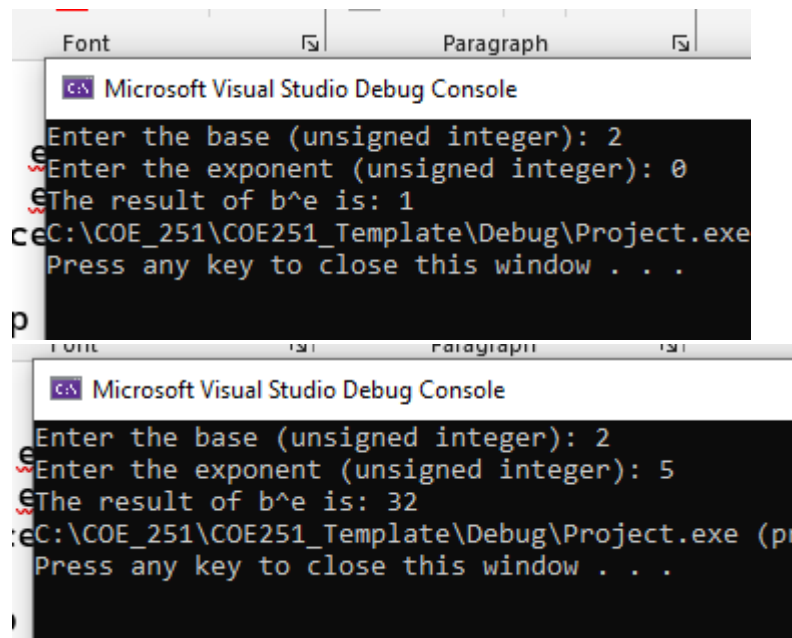
```
notzero:
    mov ebx, b
    mov ecx, e
    process:
        mul ebx
    loop process
```

```
iszero:
    mov result, eax
    pop eax
    pop ecx
    ret
```

Power ENDP

END main

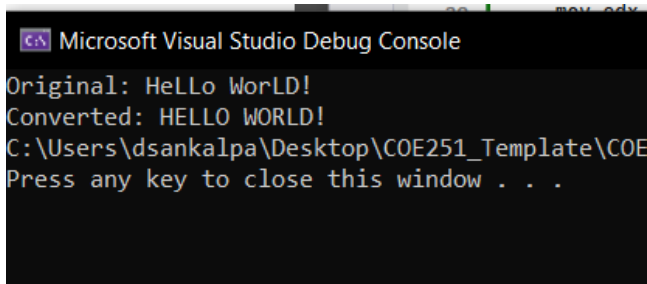
Screenshot:



3. Write an assembly program that uses two procedures called **CheckLowercase** and **ConvertToUppercase**. The program should take a null-terminated string defined in the data segment and convert all lowercase letters in the string to uppercase. The program must use the following two procedures and display the result in the **MAIN**:

- **CheckLowercase**: Takes a character (in **AL**) and checks if it is between 'a' and 'z'. Returns 1 in **EAX** if it is lowercase, otherwise returns 0.
- **ConvertToUppercase**: Takes a character (in **AL**) and converts it to uppercase by subtracting 32. The result is returned in **AL**.

Use the following string: `str_org BYTE "HeLLo WorLD!" , 0`



Solution:

TITLE Name/Purpose Here

INCLUDE Irvine32.inc

.data

```
str_org BYTE "HeLlO WorLD!",0
org1 BYTE "Original: ",0
conv BYTE "Converted: ",0
```

.code

main PROC

```
mov edx, OFFSET org1
call writestring
mov edx, OFFSET str_org
call writestring

mov ecx, lengthof str_org
mov esi, 0
checking:
    mov al, str_org[esi]
    call CheckLowercase
    cmp eax, 0
    je skip
    mov al, str_org[esi]
    call ConvertToUppercase
    mov str_org[esi], al
    skip:
        add esi, 1
```

loop checking

```
call crlf
mov edx, OFFSET conv
call writestring
```

```
mov edx, OFFSET str_org
call writestring
```

exit

main ENDP

CheckLowercase PROC

```
cmp al, 'a'
jb skip
```

```
cmp al, 'z'
```

```

ja skip

mov eax, 1
ret

skip:
    mov eax, 0
    ret

```

CheckLowercase ENDP

ConvertToUppercase PROC

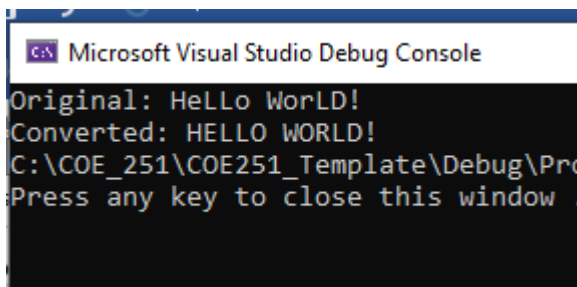
```
sub al, 32
```

```
ret
```

ConvertToUppercase ENDP

END main

Screenshot:



4. Write an assembly program that simulates a simplified version of the game Wordle, where the user attempts to guess a 5-letter word in up to 5 attempts.

The following are the game rules:

- The target word "PLANE" is hardcoded in memory.
- The user is prompted to enter a 5-letter uppercase word on each attempt.
- The program compares each guessed letter against the target word and provides feedback:
 - [x] if the letter is correct and in the correct position
 - (x) if the letter is in the word but in a different position
 - x if the letter is not in the word
- The user has 5 attempts to guess the word correctly.
- The program terminates early (**based on the winFlag**) if the user guesses the word successfully.
- At the end of the game, display one of:
 - "Correct! You guessed the word." if guessed correctly
 - "Game over. The correct word was PLANE." if the user fails

Wordle				
E	N	J	O	Y
O	T	H	E	R
W	O	R	D	Y
G	A	M	E	S
T	O	D	A	Y

To simplify the design of the game, **the following assumptions can be made:**

- The user will only input **valid, existing 5-letter words in the dictionary.**
- All letters are uppercase.
- The target word and guesses do not contain duplicate letters.

To design the game, you must use the two following procedures:

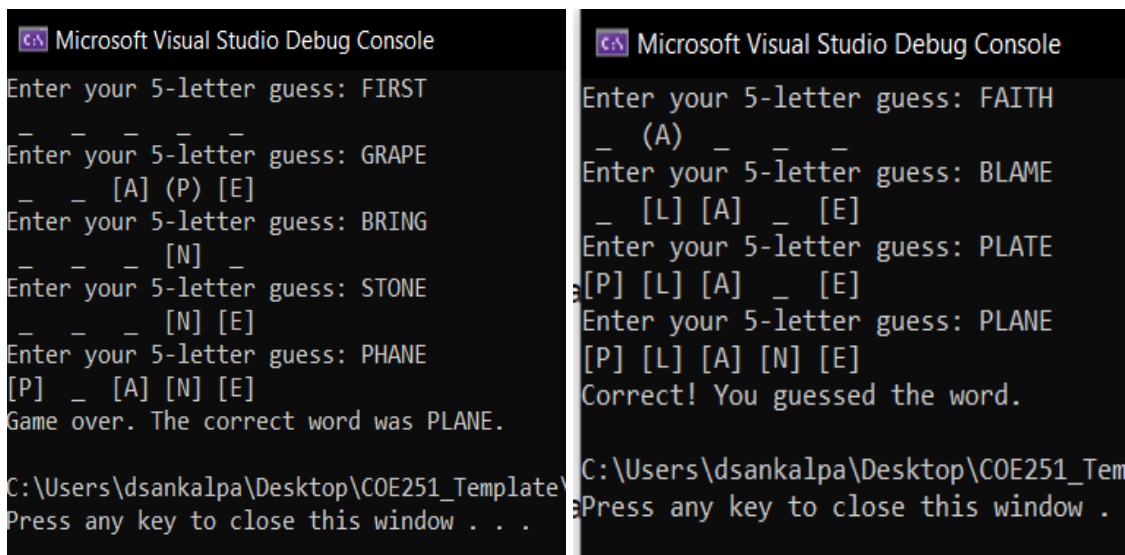
1. CompareGuess: This procedure compares the user's 5-letter guess against the target word.

- For each character:
 - If it is in the correct position → store 2 in the result array
 - If it exists elsewhere in the word → store 1
 - If it is not present in the word → store 0
- It also sets the global variable winFlag to 1 if all 5 letters match exactly, otherwise sets it to 0.

2. DisplayResult: This procedure displays each letter of the guess using visual cues:

Letters are displayed one by one with spacing for readability.

- [X] for correct position
- (X) for wrong position
- X for not in the word



```
Microsoft Visual Studio Debug Console
Enter your 5-letter guess: FIRST
_ _ _ _ _
Enter your 5-letter guess: GRAPE
_ _ [A] (P) [E]
Enter your 5-letter guess: BRING
_ _ _ [N] _
Enter your 5-letter guess: STONE
_ _ _ [N] [E]
Enter your 5-letter guess: PHANE
[P] _ [A] [N] [E]
Game over. The correct word was PLANE.
C:\Users\dsankalpa\Desktop\COE251_Template
Press any key to close this window . . .

Microsoft Visual Studio Debug Console
Enter your 5-letter guess: FAITH
_ (A) _ _ _
Enter your 5-letter guess: BLAME
_ [L] [A] _ [E]
Enter your 5-letter guess: PLATE
[P] [L] [A] _ [E]
Enter your 5-letter guess: PLANE
[P] [L] [A] [N] [E]
Correct! You guessed the word.
C:\Users\dsankalpa\Desktop\COE251_Template
Press any key to close this window . . .
```

ReadString (Irvine 32): Reads a string from the user and stores it in memory.

- EDX → address of the buffer / string array (usually of length MAX_STR_LEN + 1)
- ECX → max number of characters (including null terminator)
- Adds a null byte at the end

```
mov edx, OFFSET buffer
mov ecx, 6 ; read 5 characters + null
```


call ReadString

Solution:

TITLE Wordle Game

INCLUDE Irvine32.inc

.data

target BYTE "PLANE",0

prompt BYTE "Enter your 5-letter guess: ",0

winn BYTE "Correct! You guessed the word.",0

lostt BYTE "Game over. The correct word was PLANE.",0

guess BYTE 6 DUP(?)

result BYTE 5 DUP(?)

winFlag DWORD 0

.code

main PROC

mov ecx, 5

game_loop:

mov edx, OFFSET prompt

call WriteString

push ecx

mov edx, OFFSET guess

mov ecx, 6

call ReadString

call CompareGuess

call DisplayResult

call Crlf

cmp winFlag, 1

je won

pop ecx

loop game_loop

mov edx, OFFSET lostt

call WriteString

jmp done

won:

pop ecx

mov edx, OFFSET winn

call WriteString

done:

exit

main ENDP

CompareGuess PROC

mov winFlag, 1

mov ecx, 5

```

    mov esi, 0

compare_loop:
    mov al, guess[esi]
    cmp al, target[esi]
    je correct_position

    mov winFlag, 0

    push ecx
    mov ecx, 5
    mov edi, 0
check1:
    cmp al, target[edi]
    je exists1
    inc edi
    loop check1

    mov result[esi], 0
    pop ecx
    jmp next

exists1:
    mov result[esi], 1
    pop ecx
    jmp next

correct_position:
    mov result[esi], 2

next:
    inc esi
    loop compare_loop
    ret
CompareGuess ENDP

DisplayResult PROC
    mov ecx, 5
    mov esi, 0

display:
    mov al, ' '
    call WriteChar

    cmp result[esi], 2
    je correcttt
    cmp result[esi], 1
    je wrongg

    mov al, '_'
    call WriteChar
    jmp space

correcttt:

```

```

mov al, '['
call WriteChar
mov al, guess[esi]
call WriteChar
mov al, ']'
call WriteChar
jmp next1

```

wrongg:

```

mov al, '('
call WriteChar
mov al, guess[esi]
call WriteChar
mov al, ')'
call WriteChar

```

next1:

```

mov al, ' '
call WriteChar

```

space:

```

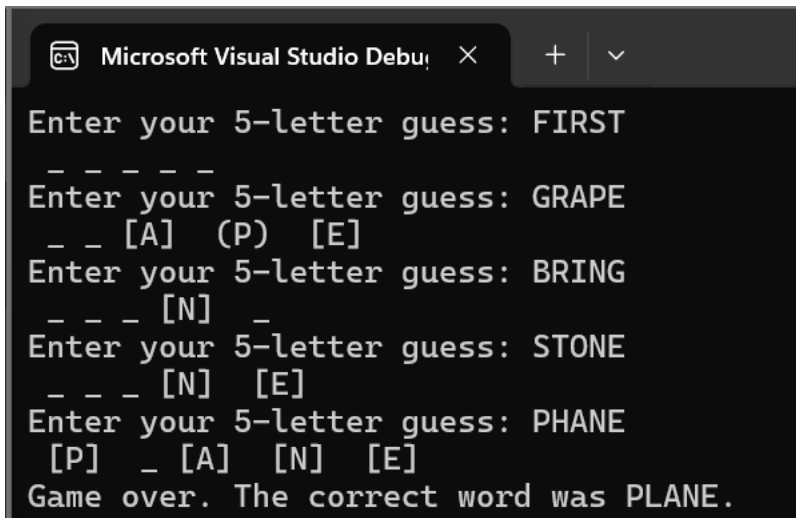
inc esi
loop display
ret

```

DisplayResult ENDP

END main

Screenshot:



```

Microsoft Visual Studio Debug Console
Enter your 5-letter guess: FIRST
_ _ _ _ _
Enter your 5-letter guess: GRAPE
_ _ [A] (P) [E]
Enter your 5-letter guess: BRING
_ _ _ [N] _
Enter your 5-letter guess: STONE
_ _ _ [N] [E]
Enter your 5-letter guess: PHANE
[P] _ [A] [N] [E]
Game over. The correct word was PLANE.

```

```
Microsoft Visual Studio Debug Console
Enter your 5-letter guess: FAITH
_ (A) _ _ _
Enter your 5-letter guess: BLAME
_ [L] [A] _ [E]
Enter your 5-letter guess: PLATE
[P] [L] [A] _ [E]
Enter your 5-letter guess: PLANE
[P] [L] [A] [N] [E]
Correct! You guessed the word.
```